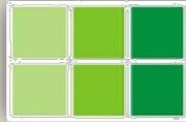




Important: The information contained in this document is for exclusive use of Goa Electronics Limited. The information contained in this document should not be passed on to any third party in part or full.

**REVISION HISTORY**

Revision No. /Version No.	Date	Prepared by	Approved by	Details of Change
1.0	17-Feb-2017	Sagar Jaiwar(Assistant Manager)	Revati Mujumdar(CEO)	-
2.0	17-Aug-2017	Anant Yende(CTO)	Revati Mujumdar(CEO)	Additional standards and Revisions
3.0	11-Jul-2024	Austin De Sa (Senior Software Developer)	Anant Yende(CTO)	Added Code Re-Usability & Modularity, Code Review and Best Practices

**TABLE OF CONTENTS**

Sr. No	Title	Page No
1	Introduction	4
2	Code Structure and Formatting	4
3	Naming Conventions	4
4	Code Documentation	5
5	Error Handling	5
6	Code Re-usability and Modularity	6
7	Testing	7
8	Security	7
9	Code Review	8
10	Documentation and commenting	8
11	Third party libraries / References	8
12	Web Server & API Standards	9
13	Best Practices	10
14	ANNEXURE 'A' - Code Review Checklist	12



1. Introduction

This document outlines the general coding standards to be adhered by all developers within the organization to ensure consistency, readability, maintainability, and quality of code.

2. Code Structure and Formatting

2.1 Indentation

- Proper and consistent indentation is important in producing easy to read and maintainable programs.
- Use tab for indentation. Use 1 tab for each level.
- Emphasize the body of a conditional statement.

2.2 Line Length

Limit lines to 80 characters. If longer lines are necessary, ensure readability.

2.3 Braces

Place opening braces on the same line as the statement. Closing braces should be on a new line. Example:

```
java
if (condition) {
    // code
} else {
    // code
}
```

2.4 Spacing

Use spaces around operators (e.g., `+`, `-`, `=`, `==`) and after commas.

2.5 Comments

Use single-line comments (`//`) for short explanations. Use block comments (`/\* ... \*/`) for longer descriptions or disabling code. Add comments to describe the purpose of classes, methods, and complex code blocks.

3. Naming Conventions

3.1 Variables

- Use camelCase [<dataType><VariableName>]
- While defining, use datatype along with variable name to give clarity about datatype and variable in subsequent references in the code. eg. strName, dtBirth, numAmt etc.



3.2 Constants

Use UPPER_CASE with underscores (e.g., `MAX\_VALUE`).

3.3 Functions/Methods

- Use camelCase (e.g., `calculateTotal()`).
- The Function/Method name should clearly outline the purpose its written for.

3.4 Classes

Use PascalCase (e.g., `CustomerOrder`).

3.5 Code Files

- Match the class name if the file contains a single class. (eg. customerorder.js).
- Use lowercase with hyphens for other files (e.g., `order-processor.js`).

3.6 Don't s

- Do not use names that begin with a numeric character.
- Do not use reserved keywords as names.

4. Code Documentation

4.1 Documentation Comments

Use Javadoc (or equivalent) comments for classes, methods, and complex code sections.

```
java
/**
 * Calculates the total price of the order.
 *
 * @param items List of items in the order
 * @return Total price
 */
public double calculateTotal(List<Item> items) {
    // method implementation
}
```



5. Error Handling

5.1 Exceptions

Catch specific exceptions rather than a generic `Exception`. Always clean up resources (e.g., close files) in a `finally` block or use try-with-resources where available.

```
java
try {
    // code that might throw an exception
} catch (Specific-exception e) {
    // handle exception
} finally {
    // cleanup code
}
```

- Use generic error messages.
- Avoid revealing error detail and internal exceptions messages to user.

5.2 Logging

- Log errors with sufficient detail to aid in debugging.
- Don't log the binary data or huge data without approval from system architect.
- Never declare an empty catch block.
- Avoid nesting a try/catch within a catch block.
- Only use the finally block to release resources from a try statement.
- Do not use try/catch blocks for flow-control.
- Avoid re throwing an exception. Allow it to bubble-up instead.

6. Code Re-usability and Modularity

6.1 Functions/Methods

- Write small, single-purpose functions.
- Avoid large functions.
- Maximum 50 lines of code in a single function.



6.2 DRY Principle

- Do not Repeat Yourself.
- Extract common code into reusable functions or modules.

6.3 Modularity

Organize code into coherent modules, classes, and packages.

7. Testing

7.1 Unit Tests

- Write unit tests for all critical functionalities.
- Test code before submitting for testing.

7.2 Integration Tests

- Ensure components work together as expected.
- Automated security testing.

8. Security

8.1 Input Validation

- Validate all inputs to prevent injection attacks.
- Whitelisting to define accepting input values, rejecting anything not explicitly defined.

8.2 Data Encryption / Protection

- Encrypt sensitive data at rest and in transit using AES.
- Password encryption using MD5.
- Do not store credentials in open text or in configuration / properties file in code.
- During login page implementation the password must always be appended with salt key.
- Store sensitive data in secure locations, avoid hard-coding credentials and secrets in the code.

8.3 Authentication and Authorization

- Use secure methods to handle authentication and authorization.
- User Role based access control.
- Wherever possible use multi-factor authentication if within scope.
- Securely manage sessions including timeout and prevent session hijacking.



8.4 Secure Communication

- Use latest version of TLS
- Use SSL Certificate with 2048 bit key, wherever possible otherwise 1024 bit
- Client side and Server-side validation should be implemented, to prevent Cross Site Scripting (XSS) vulnerability
- Token should be generated and validated on every request which is made to prevent Cross Site Request Forgery (CSRF) vulnerability

8.5 Secure Web Server

- Except GET & POST, all methods should be disabled on the Web Server.
- SameSite and Secure flag should be set on cookie.
- Application Session timeout must be set to 15 minutes.
- Ensure Security Headers compliance. Refer <https://www.ssllabs.com/ssltest/>
- Conduct 3rd party security audit from CERTIN empanelled vendor.
- Mandatory carry out VAPT of the server.

9. Code Review

9.1 Peer Review

- All code changes must be reviewed by at least one other developer.
- Follow code review checklist while reviewing the code snippets as per ANNEXURE 'A'.

10. Documentation and Commenting

10.1 Inline/Multi-line Comments

- Use inline comments to explain complex logic in short.
- Use multi-line comments to describe in more detail for complex functions.

10.2 Method Documentation

Each method should have a comment explaining its purpose, parameters, and return value.

10.3 Class Documentation

Code file (HTML, CSS, JavaScript, Java, CS etc) and class files

Must clearly state

Author - _____

Created On - _____



Purpose - _____

Any alterations in these in future date must be mentioned on top of the page.

Modified on date - Modified date & modified by

Modification done

And at respective code block

Modified on _____ by _____

11. Third party libraries / References

- Validate source using code signing or integrity details.
- Keep the copies of external JS and CSS file locally on the server instead of pointing to external source.
- Approval from System Architect / CTO mandatory if using any 3rd party libraries or external URLs.

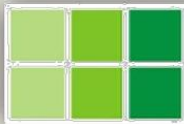
12. Web Service & API Standards

Any API's developed must have following checks implemented

- Implement rate limiting to avoid data theft and repetitive calls on API
- Define encryption procedures
- Each web method should have clear definition of input and output parameters
- Validate input before processing
- Each web API request need to be authenticated
- Any error data to be logged and alerted to the caller
- Implement token-based authentication
- Whitelist caller wherever possible
- The output/response of the API should always have an errorYN, error message and data
- The errorYN should Y if there is any error and the error message has the error info, and the data is blank in case of errorYN is Y
- If errorYN is N that means no error and the error message is blank and, in the data, the specified data is provided
- Log the API access in a notepad or DB table

Note: - The SOAP based web server are to be avoided and if really needed to be done them then approval of system architect/CTO is a must before doing so.

Preference to be given to web API.



- 12.1 Must be modular and reusable.
- 12.2 Scalable - API must be scalable and can handle increased load.
- 12.3 Implementing authentication & authorization and encryption to protect data.
- 12.4 Provide comprehensive and clear documentation for each API
- 12.5 Follow REST principles using appropriate HTTP methods (GET, POST, PUT, DELETE) and status codes.



13. Best Practices

13.1 Write comments and documentation

Perhaps one of the first things you learn as a developer is to comment your code. At first it may seem like a waste of time, following the mentality of 'If they are a developer too – they can understand it'. While it is true some of the time, commenting your code and providing proper documentation will guide the other developers through the algorithm and logic that you implemented. But do not get carried away and comment every line of code! Obvious code should be left as is.

13.2 Write readable yet efficient code

Readable codes are easy to follow, yet use optimal space and time. When writing code, you may often want to write it in as little lines as possible. Perhaps you can write an entire method or function in one line, but that only makes it harder to read and understand.

13.3 Use helper methods

It's good practice to keep code concise and succinct. A method should only implement what it needs to do. If a part of your implementation does something else, put it in a separate method and call that within your method.

13.4 If avoidable, do NOT hard-code!

The only things that should be hard-coded are constants. That's it.

13.5 Write test cases. Don't forget the edge cases: 0s, empty strings/lists, nulls, etc.

This way you know what your method does, what it takes and what it should return. You will know when it should work or when it should fail. A function should always be based on test cases; not tests on functions.

13.6 Write readable yet efficient code. Conform to the coding standards of your current project

"DISCIPLINED PERSONAL PRACTICES CAN REDUCE DEFECT INTRODUCTION RATES BY UP TO 75 PERCENT"

There are IDE's where you can set the preferred style, which will auto-correct you when you save. It is easier to read and, therefore understand, when all the files of the projects use the same style, naming convention, spacing, etc.

13.7 Use your IDE's drop-down menu

Not so much a standard, but a good practice, these are very helpful and will help you make less coding errors. For example: If you want to rename a variable or a method, just use the "refactor" option of your IDE and it will change all the calls to that method/variable name. You don't have to change them one by



one which will make your code prone to error/failure. If you want to create getter/setter methods of all your private variables, just use "create getter/setter" and the IDE will automatically create the methods for you.

13.8 APIs are handy

"CURRENT SOFTWARE PROJECTS SPEND ABOUT 40 TO 50 PERCENT OF THEIR EFFORT ON AVOIDABLE REWORK"

Before implementing a method, make sure they're not available for you to use. You don't have to code if you can just import the functionality. It makes a developer's life super easy. The famous saying is "Don't reinvent the wheel" and it stands true in many cases. However, you should always consider the implications of importing a library, especially if it is third party. Aside from possible licensing issues, you may just be bloating your codebase. If all you need is a method that converts temperatures, you don't have to import a library that does that and a hundred other things.

13.9. Pair programming/code review

"PEER REVIEWS CATCH 60 PERCENT OF THE DEFECTS"

These are very helpful when it comes to refactoring your code. Others might see better implementation to optimize your code or just make your code more elegant. It also ensures that developers adhere to standards and the work is double checked. In addition to all that, it is a wonderful way for developers to learn from each other.

13.10. Backup and save your work often

Enough said. Dead battery, blackout, software glitch, fire, nuclear disaster – all of these may result in loss of data. Making sure you save often and back up your code on some kind of version control system is a simple way to ensure that your code stays safe.

13.11. Backup and save your work often

Peer Reviews:

Conduct code reviews for all significant changes. Ensure that code is reviewed by at least one other developer.

Review Checklist:

Use a checklist to ensure consistency in code reviews, focusing on correctness, readability, security and performance.



ANNEXURE 'A' - Code Review Checklist

1. Code Structure and Format

1.1 Indentation

1.2 Comments

2. Naming Conventions

3. Code Documentation

4. Form Handling

5. Function

5.1 Length

5.2 Simple purpose

5.3 Repetitive

6. Unit Test Written

7. Security

7.1 I/P validation

7.2 Data encryption

7.3 Session token validation